

DBMS PROJECT REPORT

SELLER AND INVENTORY MANAGEMENT SYSTEM

1. Scenario

The e-commerce company sells products supplied by multiple sellers. The system should maintain seller details, track the products supplied by each seller, and monitor inventory levels to ensure product availability.

The system consists of **Seller, Product, and Inventory** information. Seller details are maintained separately, while inventory information is maintained for each product.

The main requirements are:

1. Create **Seller and Inventory Tables**.
2. Establish relationships between Seller, Product, and Inventory.
3. Add Primary Keys to identify records uniquely.
4. Add Foreign Keys to establish relationships between tables.
5. Apply suitable constraints to maintain data integrity.
6. Insert seller and inventory records.
7. Generate reports related to sellers and inventory.

2. Create Seller and Inventory Tables

Two important tables are created for the system:

Seller Table

The Seller Table stores information about the sellers who supply products.

It contains seller-related details such as:

- Seller ID
- Seller Name
- Email
- Phone Number
- Address

Inventory Table

The Inventory Table stores information about the stock available for products.

It contains information such as:

- Inventory ID
- Product ID
- Stock Quantity

The tables are created using SQL commands with appropriate data types.

Seller table Creation:

```
SQL> CREATE TABLE Seller(Seller_ID VARCHAR(7),Seller_Name VARCHAR(25),Email VARCHAR(30),Phone NUMBER(10),Address VARCHAR(45));  
Table created.
```

Inventory table Creation:

```
SQL> CREATE TABLE Inventory(Inventory_ID VARCHAR(7),Product_ID VARCHAR(7),Seller_ID VARCHAR(7),Stock_Quantity NUMBER,Stock_Status VARCHAR(11),Last_Updated DATE);  
Table created.
```

3. Establish Relationships

Relationships are established between the tables to connect sellers, products, and inventory information.

3.1 Seller and Product

One seller can supply multiple products.

One product can be supplied by one or multiple sellers.

Relationship:

Seller → Product is one-to-many

This relationship helps the system identify which products are supplied by each seller.

3.2 Product and Inventory

Each product has inventory information.

The Inventory Table tracks the available stock for each product.

Relationship:

Product → Inventory is one-to-one

This relationship allows the system to maintain inventory information for each product.

The reference PDF specifies these same relationships for the Seller/Product and Product/Inventory parts of the system.

4. Primary Key

Primary keys are added to uniquely identify records in the tables.

A **Primary Key** ensures that each record has a unique identification value.

The Seller Table and Inventory Table have their respective primary keys.

For example:

- **Seller_ID** – Primary Key in Seller Table
- **Inventory_ID** – Primary Key in Inventory Table

```
SQL> ALTER TABLE Seller ADD CONSTRAINT PK_Seller PRIMARY KEY(Seller_ID);  
Table altered.
```

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT PK_Inv PRIMARY KEY(Inventory_ID);  
Table altered.
```

5. Foreign Key

A **Foreign Key** is used to establish a relationship between two tables.

The Inventory Table contains a **Product_ID** that refers to the Product Table.

This allows inventory information to be connected with the corresponding product.

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT FK_Prod FOREIGN KEY(Product_ID) REFERENCES Product(Product_ID);  
Table altered.
```

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT FK_Seller FOREIGN KEY(Seller_ID) REFERENCES Seller(Seller_ID);  
Table altered.
```

6. Constraints

Constraints are used to maintain accuracy and integrity in the database.

1. UNIQUE Email ID

A UNIQUE constraint is applied to the seller's email ID.

This ensures that the same email ID cannot be entered for multiple sellers.

```
SQL> ALTER TABLE Seller ADD CONSTRAINT Un_mail UNIQUE(Email);  
Table altered.
```

2. NOT NULL Email

The Email column is made **NOT NULL** so that an email ID must be provided when adding a seller.

```
SQL> ALTER TABLE Seller MODIFY Email NOT NULL;  
Table altered.
```

3. Positive Stock Quantity

A CHECK constraint is added to ensure that the stock quantity is positive.

This prevents invalid negative stock values from being entered into the Inventory Table.

The reference specifically includes a UNIQUE email constraint, NOT NULL email requirement, and a check that stock quantity is positive.

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT CK_Quan CHECK(Stock_Quantity>=0);  
Table altered.
```

7. Insert Values into Seller Table

After creating the Seller Table and applying the required constraints, seller records are inserted into the database.

The Seller Table contains information such as:

- Seller ID
- Seller Name
- Email
- Phone Number
- Address

.Seller Table Insertion:

```
SQL> INSERT INTO Seller Values ('&Seller_id','&Seller_Name','&email','&Phone','&Address');
Enter value for seller_id: S001
Enter value for seller_name: Arjun
Enter value for email: arjun@gmail.com
Enter value for phone: 9876543212
Enter value for address: Salem
old 1: INSERT INTO Seller Values ('&Seller_id','&Seller_Name','&email','&Phone','&Address')
new 1: INSERT INTO Seller Values ('S001','Arjun','arjun@gmail.com',9876543212,'Salem')
```

1 row created.

```
SQL> /
Enter value for seller_id: S002
Enter value for seller_name: Priya
Enter value for email: priya@gmail.com
Enter value for phone: 9876543213
Enter value for address: madurai
old 1: INSERT INTO Seller Values ('&Seller_id','&Seller_Name','&email','&Phone','&Address')
new 1: INSERT INTO Seller Values ('S002','Priya','priya@gmail.com',9876543213,'madurai')
```

1 row created.

```
SQL> /
Enter value for seller_id: S003
Enter value for seller_name: Karthik
Enter value for email: karthik@gmail.com
Enter value for phone: 9876543214
Enter value for address: Trichy
old 1: INSERT INTO Seller Values ('&Seller_id','&Seller_Name','&email','&Phone','&Address')
new 1: INSERT INTO Seller Values ('S003','Karthik','karthik@gmail.com',9876543214,'Trichy')
```

1 row created.

```
SQL> /
Enter value for seller_id: S004
Enter value for seller_name: Meena
Enter value for email: meena@gmail.com
Enter value for phone: 9876543215
Enter value for address: Erode
old 1: INSERT INTO Seller Values ('&Seller_id','&Seller_Name','&email','&Phone','&Address')
new 1: INSERT INTO Seller Values ('S004','Meena','meena@gmail.com',9876543215,'Erode')
```

1 row created.

```
SQL> /
Enter value for seller_id: S005
Enter value for seller_name: Vijay
Enter value for email: vijay@gmail.com
Enter value for phone: 9876543216
Enter value for address: Vellore
old 1: INSERT INTO Seller Values ('&Seller_id','&Seller_Name','&email','&Phone','&Address')
new 1: INSERT INTO Seller Values ('S005','Vijay','vijay@gmail.com',9876543216,'Vellore')
```

1 row created.

Seller table after Insertion:

```
SQL> SELECT * FROM Seller;
```

SELLR_I	SELLER_NAME	EMAIL	PHONE
S001	Arjun	arjun@gmail.com	9876543211
S002	Priya	priya@gmail.com	987654312
S003	Karthik	karthik@gmail.com	9876543213
S004	Meena	meena@gmail.com	9876543214
S005	Vijay	vijay@gmail.com	9876543215

8. Insert Values into Inventory Table

Inventory records are inserted after creating the Inventory Table.

The Inventory Table contains information about the stock available for each product.

The stock quantity is maintained to monitor product availability.

Inventory Table Insertion:

```
SQL> INSERT INTO Inventory VALUES('&Inventory_Id','&Product_Id','&Sellr_Id',&Stock,'&Stock_Status',To_DATE('&Last_Update','YYYY-MM-DD'));
Enter value for inventory_id: INV001
Enter value for product_id: P01
Enter value for sellr_id: S001
Enter value for stock: 4
Enter value for stock_status: Available
Enter value for last_update: 2026-09-01
old 1: INSERT INTO Inventory VALUES('&Inventory_Id','&Product_Id','&Sellr_Id',&Stock,'&Stock_Status',To_DATE('&Last_Update','YYYY-MM-DD'))
new 1: INSERT INTO Inventory VALUES('INV001','P01','S001',4,'Available',To_DATE('2026-09-01','YYYY-MM-DD'))

1 row created.

SQL> /
Enter value for inventory_id: INV002
Enter value for product_id: P02
Enter value for sellr_id: S002
Enter value for stock: 7
Enter value for stock_status: Available
Enter value for last_update: 2026-08-07
old 1: INSERT INTO Inventory VALUES('&Inventory_Id','&Product_Id','&Sellr_Id',&Stock,'&Stock_Status',To_DATE('&Last_Update','YYYY-MM-DD'))
new 1: INSERT INTO Inventory VALUES('INV002','P02','S002',7,'Available',To_DATE('2026-08-07','YYYY-MM-DD'))

1 row created.

SQL> /
Enter value for inventory_id: INV003
Enter value for product_id: P03
Enter value for sellr_id: S001
Enter value for stock: 8
Enter value for stock_status: Available
Enter value for last_update: 2026-05-29
old 1: INSERT INTO Inventory VALUES('&Inventory_Id','&Product_Id','&Sellr_Id',&Stock,'&Stock_Status',To_DATE('&Last_Update','YYYY-MM-DD'))
new 1: INSERT INTO Inventory VALUES('INV003','P03','S001',8,'Available',To_DATE('2026-05-29','YYYY-MM-DD'))

1 row created.

SQL> /
Enter value for inventory_id: INV004
Enter value for product_id: P04
Enter value for sellr_id: S004
Enter value for stock: 3
Enter value for stock_status: Available
Enter value for last_update: 2026-07-15
old 1: INSERT INTO Inventory VALUES('&Inventory_Id','&Product_Id','&Sellr_Id',&Stock,'&Stock_Status',To_DATE('&Last_Update','YYYY-MM-DD'))
new 1: INSERT INTO Inventory VALUES('INV004','P04','S004',3,'Available',To_DATE('2026-07-15','YYYY-MM-DD'))

1 row created.
```

```
SQL> /
Enter value for inventory_id: INV005
Enter value for product_id: P04
Enter value for sellr_id: S005
Enter value for stock: 8
Enter value for stock_status: Available
Enter value for last_update: 2026-09-19
old 1: INSERT INTO Inventory VALUES('&Inventory_Id','&Product_Id','&Sellr_Id',&Stock,'&Stock_Status',To_DATE('&Last_Update','YYYY-MM-DD'))
new 1: INSERT INTO Inventory VALUES('INV005','P04','S005',8,'Available',To_DATE('2026-09-19','YYYY-MM-DD'))

1 row created.
```

Inventory table after Insertion:

```
SQL> SELECT * FROM Inventory;
```

INVENTO	PRODUCT	SELLER_	STOCK_QUANTITY	STOCK_STATU	LAST_UPDA
-----	-----	-----	-----	-----	-----
INV001	P01	S001	4	Available	01-SEP-26
INV002	P02	S002	7	Available	07-AUG-26
INV003	P03	S001	8	Available	29-MAY-26
INV004	P04	S004	3	Available	15-JUL-26
INV005	P04	S005	8	Available	19-SEP-26

9. Report 1: Seller-wise Product Report

The Seller-wise Product Report provides information about products supplied by each seller.

Display:

- Seller Name
- Number of Products Supplied
- Available Stock

Joins can be used to retrieve information from the related tables.

This report helps identify the products supplied by each seller and their available stock.

```
SQL> SELECT s.Seller_Name,  
2         COUNT(i.Product_Id) AS Number_of_Products,  
3         SUM(i.Stock_Quantity) AS Available_Stock  
4 FROM Seller s  
5 JOIN Inventory i  
6 ON s.Sellr_Id = i.Seller_Id  
7 WHERE i.Stock_Status = 'Available'  
8 GROUP BY s.Seller_Name;
```

SELLER_NAME	NUMBER_OF_PRODUCTS	AVAILABLE_STOCK
-----	-----	-----
Meena	1	3
Arjun	2	12
Priya	1	7
Vijay	1	8

10. Report 2: Stock Availability Report

The Stock Availability Report shows the current stock information of products.

Display:

- Product Name
- Stock Quantity
- Stock Status

The stock status can be used to indicate whether a product is available or out of stock.

This report helps monitor product availability.

```
SQL> SELECT p.Product_Name, i.Stock_Quantity, i.Stock_Status FROM Product p JOIN Inventory i on p.product_Id = i.Product_Id;
```

```
PRODUCT_NAME
```

```
STOCK_QUANTITY STOCK_STATU
```

```
Mixer Grinder
```

```
4 Available
```

```
Electric kettle
```

```
7 Available
```

```
Face cream
```

```
8 Available
```

```
PRODUCT_NAME
```

```
STOCK_QUANTITY STOCK_STATU
```

```
Basmathi rice
```

```
3 Available
```

```
Basmathi rice
```

```
8 Available
```

11. Report 3: Inventory Analysis

Inventory Analysis is used to analyze the stock information stored in the database.

The following information can be calculated:

- **Total Products Available**

```
SQL> SELECT COUNT(*) AS Total_Available_Product FROM Inventory WHERE Stock_Status = 'Available';

TOTAL_AVAILABLE_PRODUCT
-----
                        5
```

- **Products Out of Stock**

```
SQL> SELECT COUNT(*) AS Out_of_Stock FROM Inventory WHERE Stock_Status = 'Out of Stock';

OUT_OF_STOCK
-----
            0
```

- **Highest Stocked Products**

```
SQL> SELECT p.Product_Name, i.Stock_Quantity FROM Product p JOIN Inventory i
ON p.Product_Id = i.Product_Id WHERE i.Stock_Quantity = (SELECT MAX(Stock_Quantity) FROM Inventory);

PRODUCT_NAME
-----
-----
STOCK_QUANTITY
-----
Face cream           8
Basmathi rice        8
```

- **Average Inventory Quantity**

```
SQL> SELECT AVG(Stock_Quantity) AS Average_Inventory FROM Inventory;

AVERAGE_INVENTORY
-----
                    6
```

SQL aggregate functions and queries can be used to perform this analysis.

12. Data Integrity

Data integrity is maintained using **Primary Keys, Foreign Keys, UNIQUE, NOT NULL, and CHECK constraints**.

The Primary Key uniquely identifies each record.

The Foreign Key connects related tables.

The UNIQUE constraint prevents duplicate email IDs.

The NOT NULL constraint ensures that required information is entered.

The CHECK constraint ensures that stock quantity contains valid positive values.

These constraints help maintain accurate and consistent data in the Seller and Inventory Management System.

13. Conclusion

The **Seller and Inventory Management System** provides an organized way to manage seller information and monitor product inventory.

The project includes creating Seller and Inventory Tables, establishing relationships, adding Primary and Foreign Keys, applying constraints, inserting records, and generating different reports.

The Seller-wise Product Report provides seller and product information, while the Stock Availability Report helps monitor product stock. The Inventory Analysis provides useful information such as total available products, out-of-stock products, highest stocked products, and average inventory quantity.

Thus, the project demonstrates how a **Database Management System (DBMS)** can be used to efficiently manage sellers, products, and inventory information.